ASSESSMENT- CO5-AT1-Constraint-Based Problem Solving

← Back

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

C RUTHIKA REDDY
192421138RA

1/1 answered

edge independence. Analyze how constraints impact feasible solutions. Develop a backtracking-based solution. Apply pruning to eliminate conflicting edges. Justify correctness and discuss NP-Hard complexity.

Your Code

```
1class MaximumMatching:
2    def __init__(self, num_vertices, edges)
3        self.n = num_vertices
4        self.edges = edges
5        self.best_matching = []
6        self.best_size = 0
7
8    def backtrack(self, index, used, curre
9        if index == len(self.edges):
10           if len(current) > self.best_si
11               self.best_size = len(curre
12               self.best_matching = curre
13           return
14
15
16       if len(current) + (len(self.edges)
```

```
            return

        u, v = self.edges[index]

        # Include edge if both vertices are
        if not used[u] and not used[v]:
            used[u] = True
            used[v] = True
            current.append((u, v))

            self.backtrack(index + 1, used,

            current.pop()
            used[u] = False
            used[v] = False


        self.backtrack(index + 1, used, cu

    def solve(self):
        used = [False] * self.n
        self.backtrack(0, used, [])
        return self.best_matching


n, m = map(int, input().split())

edges = []
for _ in range(m):
    u, v = map(int, input().split())
    edges.append((u, v))

obj = MaximumMatching(n, edges)
result = obj.solve()

print(len(result))
for u, v in result:
    print(u, v)
```

Input

```
7 0
5 9
0 9
0 8
```

Output

0

Submitted